The Journal of Open Source Software

openpv/simshady: A JavaScript Package for Photovoltaic Yield Estimation Based on 3D Meshes

Florian Kotthoff [1,2,3¶], **Mino Estrella** [3], **Martin Großhauser** [1], **Konrad Heidler** [1], **and Korbinian Pöppel** [1]

1 OpenPV GbR, Munich, Germany **2** OFFIS Institute for Information Technology, 26121 Oldenburg, Germany **3** Chair of Renewable and Sustainable Energy Systems, Technical University of Munich, 85748 Munich, Germany ¶ Corresponding author

DOI: 10.21105/joss.10694

Software

- Review
- Repository
- Archive

Editor: Adam R. Jensen

Reviewers:

- @RDaxini
- @kbonney

Submitted: 12 July 2025
Published: 28 October 2026

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

openpv/simshady (short: `simshady`) is a JavaScript package for simulating shading and photovoltaic (PV) energy yield. It integrates local weather data and 3D objects into its shading simulation, utilizing Three.js meshes for geometric modeling. The package performs shading analysis using a WebGL-parallelized implementation of the Möller-Trumbore intersection algorithm (Möller & Trumbore, 1997). With the software, users can obtain both aggregated and spatially or temporally resolved simulated scenes of expected PV yields for further processing. `simshady` provides both a package for web development and a command-line interface (CLI) tool for running simulations locally.

Statement of need

The main goal of `simshady` is to calculate solar radiation yields with respect to shading from surrounding objects, which is relevant for calculating the yield of PV system in real-world conditions. The yield of PV installations depends on the shading from surrounding objects such as houses, trees, and terrain, which can be obtained through the ray casting algorithm implemented in `simshady`. As the software can be used through CLI and in the browser, it targets both researchers who want to do large-scale shading simulations on a server and researchers who want to build interactive solar simulation tools. Hence, `simshady` fills a gap in the area of detailed 3D shading simulation with server- and client-side applications.

State of the field

Existing software can roughly be grouped into three groups: full-suite commercial design tools, simplified yield calculators, and developer libraries.

Full-suite design tools target professional PV planners and combine 3D system modeling, component databases, and financial analysis behind a graphical user interface. HelioScope by Aurora Solar is a web-based design platform that approximates near shading by projecting drop shadows from 3D objects rather than performing full ray casting, and is offered as a paid subscription (Aurora Solar, 2026). PV*SOL premium by Valentin Software is a Windows desktop application that performs detailed sub-module shading analysis at cell level, but is restricted to interactive single-project use, meaning it requires manual, graphical user interface (GUI)-based operation for one project at a time without automation or batch processing capabilities (Valentin Software GmbH, 2026). PVsyst is the industry reference for bankable yield reports and additionally provides a separately licensed command-line tool, PVsystCLI, which is the only existing software that combines accurate 3D near shading with batch

automation, but at a cost of several thousand Swiss francs per year (PVsyst SA, 2026). The System Advisor Model (SAM) developed by the National Laboratory of the Rockies (NLR) is open source and exposes a Python wrapper (PySAM), but its 3D shading capabilities are limited compared to dedicated design tools (National Renewable Energy Laboratory, 2026b). Within the ecosystem of the open-source geoinformation software QGIS, the Urban Multi-scale Universal Predictor (UMEP) (Lindberg et al., 2018) plugin contains a tool for shading simulations on 2D digital surface models (Lindberg et al., 2015) called SEBE (short for Solar Energy on Building Envelopes).

Among the developer libraries, the Python package pvlib (Kevin S. Anderson et al., 2023; Kevin S. Anderson et al., 2026; Holmgren et al., 2018) offers a large range of functionalities. With pvlib it is possible to model shading scenarios where the whole 3D scene is not needed. Examples are the possibility to model horizon shading from far-away objects like hills, or modeling diffuse self-shading or partial module shading, to name a few. However, the topic of shading simulation with 3D objects is not included in this package. Another Python-based software that enables irradiance modeling in two dimensions is pvfactors (Anoma et al., 2017), which computes view factors between PV rows and the ground to capture row-to-row and bifacial shading, but cannot represent arbitrary 3D obstacles such as trees or buildings. SoDeLe wraps pvlib in an end-user GUI for quick residential estimates and likewise omits 3D shading (Steinacker et al., 2026). Web-based tools for solar panel simulations, such as PVGIS, PVWatts, and RETScreen, provide an accessible means for non-technical individuals to estimate energy yields based on geographic location and building geometry (Psomopoulos et al., 2015). These tools account for far-shading from the horizon or terrain, or for self-shading derived from the ground coverage ratio, but lack the capability to perform shading simulations using 3D geometries from nearby objects. RADIANCE (Ward, 1994) is a collection of tools designed for general light and shading simulation and developed in the 1990s. For solar simulation, the bifacial_radiance tool (Ayala Pelaez & Deline, 2020) exists as a Python wrapper around the original software. In combination, these software packages offer a variety of shading methods, including detailed 3D shading simulations. Table 1 summarizes the tools by the capabilities most relevant to urban-scale PV assessment.

Table 1: Comparison of representative PV simulation tools. In the shading column, 3D means that 3-dimensional objects are used to model shading, 3D (simplified) means a 3D scene is used but objects are abstracted as polygons, DSM means that Digital Surface Models (height maps) are used, 2D means that the simulation only relies on 2D abstractions, Horizon represents shading from the horizon contour, Row represents row-to-row self-shading without 3D objects, and None means no shading modeling is provided.

Tool	Platform	Cost	Open Source	Shading	Source
HelioScope	Web	Paid	No	3D	Aurora Solar (2026)
PV*SOL prem.	Desktop	Paid	No	3D	Valentin Software GmbH (2026)
PVsyst	Desktop	Paid	No	3D	PVsyst SA (2026)
bifacial_radiance	Package	Free	Yes	3D	Ayala Pelaez & Deline (2020)
SAM	Desktop	Free	Yes	3D simplified	National Renewable Energy Laboratory (2026b)
QGIS Plugin SEBE	Desktop	Free	Yes	DSM	Lindberg et al. (2015)
PVGIS	Web	Free	No	Horizon	European Commission Joint Research Centre (2026)
PVWatts	Web	Free	Yes	Row	National Renewable Energy Laboratory (2026a)
SoDeLe	Package	Free	Yes	None	Steinacker et al. (2026)

Tool	Platform	Cost	Open Source	Shading	Source
pvlib	Package	Free	Yes	2D, Horizon	Holmgren et al. (2018); Kevin S. Anderson et al. (2023); Kevin S. Anderson et al. (2026)
pvfactors	Package	Free	Yes	2D	Anoma et al. (2017)

Software design

`simshady` simulates the yield of PV systems by considering weather/climate data and shading from local 3D geometry. It is built around three core principles: (1) To run the core simulation code efficiently on the GPU, (2) to expose two different interfaces for both browser-based applications and server-based simulation, and (3) to integrate standardized data models, namely from `Three.js` and from the Wavefront OBJ file format.

Input data

In `simshady`, two major types of input data need to be provided. First, a 3D scene is built that represents the environment, consisting of primary objects for the simulation (e.g., PV panels or target buildings) and surrounding objects that may cast shadows (e.g., neighboring buildings and trees). These objects can be provided as Wavefront .obj files or as Three.js geometries. Second, weather and climate data need to be provided as input data in .json format as a discretized skydome (Górski et al., 2005; Zonca et al., 2019). This data consists of an array of sky pixels where each pixel holds one radiance value, representing the cumulative direct and diffuse component of solar radiation over a specified time interval that hits the simulated object from the direction of the sky pixel (see Figure 1 a).

Such a skydome can be obtained, for example, by using the National Solar Radiation Database (Sengupta et al., 2018). Here, for each time interval, the Direct Normal Irradiance (DNI), normalized by the solid angle, is assigned to the sky segment that contains the position of the sun at that specific instant of time. For the Diffuse Horizontal Irradiance (DHI), the isotropic sky assumption can be used. This assumption is known to be a simplification (Perez et al., 1987), as it assigns the same contribution to the resulting DHI to all sky segments. With this assumption, the summed portion L over all sky segments on a horizontal surface results in the given DHI value and L is constant, this equation can be solved to obtain the diffuse portion of radiance per sky segment

$$\mathrm{DHI} = L \int_0^{2\pi} \int_0^{\pi/2} \cos\theta \, \sin\theta \, \mathrm{d}\theta \, \mathrm{d}\phi = L \cdot 2\pi \cdot \frac{1}{2} = L\,\pi,$$

$$L = \frac{\mathrm{DHI}}{\pi}.$$

Finally, averaging both direct and diffuse parts for each sky segment over the given time period gives the radiances in $\mathrm{W m^{-2} sr^{-1}}$.

Simulation pipeline

The central `ShadingScene` class orchestrates the simulation through the following steps:

1. **Geometry pre-processing:** First, the simulation mesh is refined by recursively subdividing triangles whose longest edge exceeds a configurable threshold (default 1.0 m), ensuring a uniform spatial resolution for the PV yield calculation. Reducing this threshold results in

a larger number of smaller triangles in the mesh, and therefore a higher resolution of the simulated shading. To improve the simulation speed, a geometry filter removes shading triangles that cannot cast shadows on the simulation geometry, given the minimum solar elevation angle present in the irradiance dataset.

2. **Ray casting:** The simulation utilizes the Möller-Trumbore intersection algorithm (Möller & Trumbore, 1997) to determine if any shading objects obstruct the view between a sky segment and the main simulation geometry (see Figure 1 a). For each triangle in the simulation geometry, a shading mask is generated, indicating whether an object blocks the incident irradiance beam from the skydome surface to the triangle. The shading mask values are binary, where 0 indicates that an object shades the triangle and 1 that there is no shading.
3. **Radiance integration and yield calculation:** The radiance values from all sky segments are needed as input data. They represent the radiance in $\mathrm{Wm^{-2}sr^{-1}}$ that reaches the simulation geometry from a given sky segment. Each radiance is multiplied by its corresponding shading mask, the cosine of the angle of incidence, and the solid angle of a sky segment. Then the products are summed over all segments to obtain the irradiation received by each triangle. The incident energy is obtained by multiplying with the time covered by the input data [in h], which results in the PV yield [in $\mathrm{Wh/m^2}$] by multiplying this energy by a configurable solar-to-electricity conversion efficiency (default 15%). This efficiency includes the PV panel efficiency as well as the ratio of the area covered by PV panels to the total area.
4. **Visualization:** The computed yield values are normalized and mapped to RGB colors using a configurable colormap (see Figure 1 b). The resulting mesh carries both the color attribute for visualization and per-triangle solar yield for further analysis.

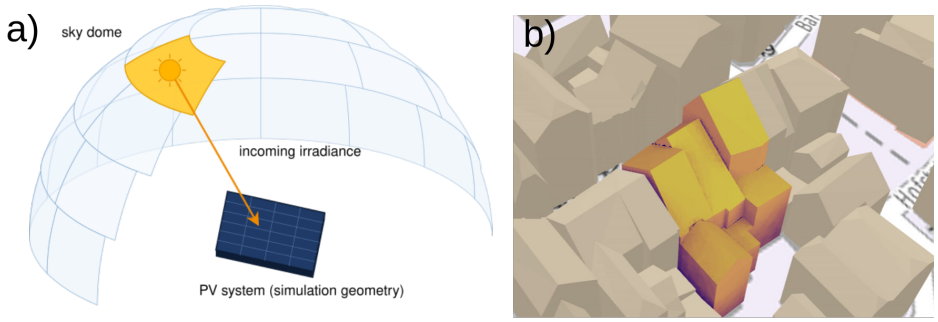


Figure 1: a) Schema of the simulation setup: For each segment of the skydome and for each triangle of the simulation geometry, it is simulated if an object blocks the incoming irradiance. b) A simulated building with its yearly averaged solar yield, where dark purple represents low yields and light yellow represents high yields. Screenshot taken from (Grosshauser et al., 2024).

Simshady in the browser

`simshady` reuses the data model and objects of `Three.js`, a common package for 3D web applications. The package processes `Three.js` meshes and adds the simulated PV yield as a color to the mesh, as shown in Figure 1 b). Additionally, each triangle of the simulated buildings has its solar yield assigned as an attribute for further processing.

Simshady in the terminal

The `simshady` CLI is a wrapper around the core WebGL-based simulation engine that enables batch processing of photovoltaic-yield analyses on a server. It first parses a small set of required arguments (the simulation geometry and the irradiance data). The supplied geometry files,

either JSON objects or Wavefront OBJ files, are handed to a headless Chromium instance launched via Puppeteer (Google Chrome Team, 2025).

Inside the browser context, the full `simshady` package is injected, the scene is reconstructed, and the GPU-accelerated Möller-Trumbore ray casting routine is executed. When the calculation finishes, the CLI extracts the mesh data from the browser and writes the following artifacts into the user-specified output directory: binary files for vertex positions, colors, and intensities; a color-coded Wavefront OBJ file with per-vertex RGB values; an orthographic top-down PNG snapshot of the scene; and a JSON summary containing per-timestep and total aggregated yield statistics such as total and mean yield or surface area.

Research impact statement

Compared to the state of the field, `simshady` fills two existing gaps: First, it is implemented in TypeScript and hence simultaneously enables simulations in the browser and on a local machine. Second, it tackles the calculation-intensive task of ray casting for shading simulation with a performant WebGL implementation on the GPU. The shader language GLSL (OpenGL Shading Language), which WebGL uses, is not familiar to most developers: it only ranks 35th among programming languages on GitHub (GitHub, 2026). Hence, `simshady` makes the parallelized simulation of shading more accessible by exposing the WebGL simulation code through the more familiar JavaScript APIs instead.

City-scale demonstration: Munich

One existing demonstration of the research impact was the application of the `simshady` CLI to simulate the PV yield for the entire city of Munich. The original data consists of 109 CityGML Level of Detail 2 (LoD2) tiles, each covering 2 km x 2 km. For processing, each tile was split into four tiles of one square kilometer each. The 3D data contained roughly 200 km² of building surface in total. Averaged incoming irradiance was derived from the National Solar Radiation Database (Sengupta et al., 2018). Figure 2 a) shows one 3D mesh output for one tile. All tiles combined generate a simulation scene of the whole city of Munich, which is depicted in Figure 2 b) as an orthographic top-down image.

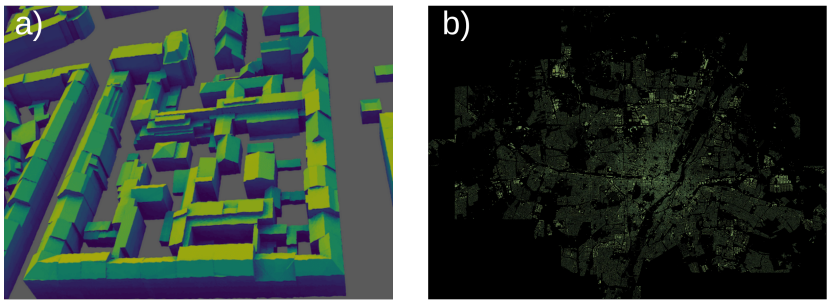


Figure 2: a) 3D mesh output of the CLI for a single Munich tile, with per-triangle annual PV yield mapped onto the building geometry (dark blue = low, light yellow = high). b) Orthographic top-down snapshots generated by the CLI for all Munich tiles, combined in one image.

The complete simulation took roughly 110 hours on a single cloud GPU server running four parallel CLI instances and cost less than 60 USD. The full output, including geometry, intensities, metadata, and visualization artifacts for all tiles, makes up roughly 35 GB in compressed format and is openly available as a dataset on Zenodo under CC BY 4.0 (Estrella & Kotthoff, 2026).

Conclusion

The `simshady` package serves two primary purposes: it provides a solution for scientific calculations of PV yield on local machines through the CLI, while also facilitating science communication through interactive web-based simulations. Two design decisions are especially relevant: By implementing the main algorithm in WebGL, the package achieves higher performance than a pure JavaScript implementation, and it offers a JavaScript wrapper around PV simulations in WebGL.

In the future, `simshady` can benefit from various developments: First, the architectural decision of outsourcing the calculation of irradiance has a clear benefit. If averaged irradiance data are provided for a whole year, `simshady` calculates the averaged PV yield per year. If irradiance time series data are provided, `simshady` can also simulate time series of PV yield that show the local shading influence at highly resolved times. However, with this benefit comes the disadvantage that users need to provide irradiance data themselves. One useful extension would be to integrate the creation of these time series into the software. Second, the interoperability with existing standards has started in `simshady`, but has not finished. Providing results in formats that can be reused in other software tools, like `pvlib` for example, would also be beneficial for the whole community.

To extend and maintain `simshady` in the future, other researchers and developers are warmly welcome to contribute to the codebase or discuss new ideas.

CRediT author statement

FK: Conceptualization, Software, Funding acquisition, Writing – original draft

MG: Conceptualization, Software, Funding acquisition, Writing – review & editing

KH: Conceptualization, Software, Funding acquisition, Writing – review & editing

ME: Software, Writing – review & editing

KP: Conceptualization, Software, Funding acquisition, Writing – review & editing

AI usage disclosure

After designing the software architecture and writing core functionalities, both open-weight and proprietary LLM-based chatbots were used to implement or debug some of the methods in `simshady`. The open-weight models gpt-oss:120b and deepseek-r1:70b were used for reviewing the written text of this paper. Individual parts of the SVG file for Figure 1 were created with the help of a proprietary LLM.

Acknowledgements

The development of this software was funded by the German Federal Ministry of Research, Technology and Space within the "Software Sprint - Support for Open Source Developers" program by the Prototype Fund.

References

Anderson, Kevin S., Hansen, C. W., Holmgren, W. F., Jensen, A. R., Mikofski, M. A., & Driesse, A. (2023). Pvlib python: 2023 project update. *Journal of Open Source Software*, *8*(92), 5994. https://doi.org/10.21105/joss.05994

Anderson, Kevin S., Jensen, A. R., & Hansen, C. W. (2026). A bifacial view factor model considering terrain slope and nonuniform albedo. *IEEE Journal of Photovoltaics*, *16*(4), 605–613. https://doi.org/10.1109/JPHOTOV.2026.3677506

Anoma, M. A., Jacob, D., Bourne, B. C., Scholl, J. A., Riley, D. M., & Hansen, C. W. (2017). View factor model and validation for bifacial PV and diffuse shade on single-axis trackers. *2017 IEEE 44th Photovoltaic Specialist Conference (PVSC)*, 1549–1554. https://doi.org/10.1109/PVSC.2017.8366704

Aurora Solar. (2026). *HelioScope solar design software*. https://helioscope.aurorasolar.com/

Ayala Pelaez, S., & Deline, C. (2020). Bifacial_radiance: A python package for modeling bifacial solar photovoltaic systems. *Journal of Open Source Software*, *5*(50), 1865. https://doi.org/10.21105/joss.01865

Estrella, M., & Kotthoff, F. (2026). *simshady munich PV yield simulation dataset (CityGML LoD2, 432 tiles)*. Zenodo. https://doi.org/10.5281/zenodo.18742162

European Commission Joint Research Centre. (2026). *PVGIS – photovoltaic geographical information system*. https://re.jrc.ec.europa.eu/pvg_tools/en/

GitHub. (2026). *GitHub Innovation Graph*. https://github.com/github/innovationgraph

Google Chrome Team. (2025). *Puppeteer: Headless chrome node.js API* (Version v24.31.0). Google. https://github.com/puppeteer/puppeteer

Górski, K. M., Hivon, E., Banday, A. J., Wandelt, B. D., Hansen, F. K., Reinecke, M., & Bartelmann, M. (2005). HEALPix: A framework for high-resolution discretization and fast analysis of data distributed on the sphere. *The Astrophysical Journal*, *622*(2), 759. https://doi.org/10.1086/427976

Grosshauser, M., Kotthoff, F., Heidler, K., & Pöppel, K. (2024). *Openpv.de*. www.openpv.de

Holmgren, W. F., Hansen, C. W., & Mikofski, M. A. (2018). Pvlib python: A python package for modeling solar energy systems. *Journal of Open Source Software*, *3*(29), 884. https://doi.org/10.21105/joss.00884

Lindberg, F., Grimmond, C. S. B., Gabey, A., Huang, B., Kent, C. W., Sun, T., Theeuwes, N. E., Järvi, L., Ward, H. C., Capel-Timms, I., Chang, Y., Jonsson, P., Krave, N., Liu, D., Meyer, D., Olofson, K. F. G., Tan, J., Wästberg, D., Xue, L., & Zhang, Z. (2018). Urban multi-scale environmental predictor (UMEP): An integrated tool for city-based climate services. *Environmental Modelling & Software*, *99*, 70–87. https://doi.org/10.1016/j.envsoft.2017.09.020

Lindberg, F., Jonsson, P., Honjo, T., & Wästberg, D. (2015). Solar energy on building envelopes – 3D modelling in a 2D environment. *Solar Energy*, *115*, 369–378. https://doi.org/10.1016/j.solener.2015.03.001

Möller, T., & Trumbore, B. (1997). Fast, minimum storage ray-triangle intersection. *Journal of Graphics Tools*, *2*(1), 21–28. https://doi.org/10.1080/10867651.1997.10487468

National Renewable Energy Laboratory. (2026a). *PVWatts calculator* (Version 8). https://pvwatts.nrel.gov

National Renewable Energy Laboratory. (2026b). *System advisor model (SAM)*. https://sam.nrel.gov

Perez, R., Seals, R., Ineichen, P., Stewart, R., & Menicucci, D. (1987). A new simplified version of the perez diffuse irradiance model for tilted surfaces. *Solar Energy*, *39*(3), 221–231. https://doi.org/10.1016/S0038-092X(87)80031-2

Psomopoulos, C. S., Ioannidis, G. C., Kaminaris, S. D., Mardikis, K. D., & Katsikas, N. G. (2015). A comparative evaluation of photovoltaic electricity production assessment

software (PVGIS, PVWatts and RETScreen). *Environmental Processes*, *2*, 175–189. https://doi.org/10.1007/s40710-015-0092-4

PVsyst SA. (2026). *PVsyst photovoltaic software* (Version 8). https://www.pvsyst.com

Sengupta, M., Xie, Y., Lopez, A., Habte, A., Maclaurin, G., & Shelby, J. (2018). The national solar radiation data base (NSRDB). *Renewable and Sustainable Energy Reviews*, *89*, 51–60. https://doi.org/10.1016/j.rser.2018.03.003

Steinacker, H., Mucke, J., Ott, E., & Stickel, M. (2026). *SoDeLe: Solarsimulation denkbar leicht*. https://github.com/QuaSi-Software/SoDeLe

Valentin Software GmbH. (2026). *PV*SOL premium – planning and simulation of photovoltaic systems*. https://valentin-software.com/en/products/pvsol-premium/

Ward, G. J. (1994). The RADIANCE lighting simulation and rendering system. *Proceedings of the 21st Annual Conference on Computer Graphics and Interactive Techniques*, 459–472. https://doi.org/10.1145/192161.192286

Zonca, A., Singer, L., Lenz, D., Reinecke, M., Rosset, C., Hivon, E., & Gorski, K. (2019). Healpy: Equal area pixelization and spherical harmonics transforms for data on the sphere in python. *Journal of Open Source Software*, *4*(35), 1298. https://doi.org/10.21105/joss.01298